PRACTICAL - 09

Aim: Counter race (Counter.java + Driver.java)

Code Screenshot:

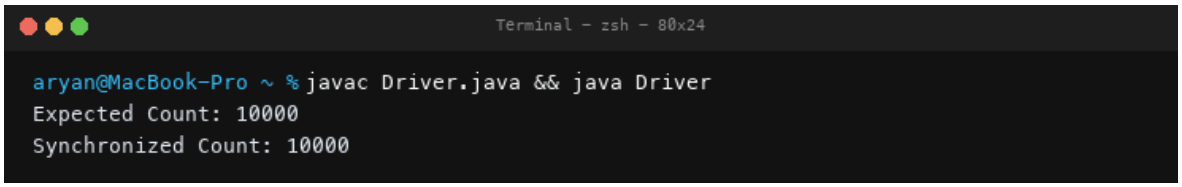
Counter.java

```java
public class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public int getCount() {
        return count;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Thread[] threads = new Thread[10];
        for (int i = 0; i < threads.length; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) counter.increment();
            });
            threads[i].start();
        }
        for (Thread t : threads) t.join();
        System.out.println("Expected Count: 10000");
        System.out.println("Synchronized Count: " + counter.getCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood race conditions and critical section vulnerabilities in Java.
- Mastered the synchronized method modifier for mutual exclusion.
- Coordinated thread completion deterministically using Thread.join().
- Analyzed memory visibility and read-modify-write atomicity guarantees.
- Verified thread safety across concurrent multi-threaded execution paths.

Aim: Seat booking race (CinemaBooking.java + Driver.java)

Code Screenshot:

CinemaBooking.java

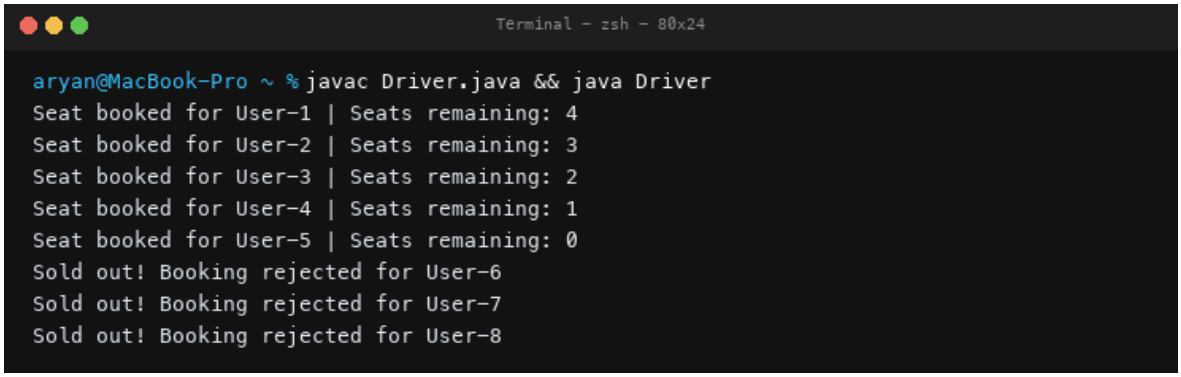
```java
public class CinemaBooking {
    private int seatsLeft = 5;

    public synchronized boolean bookSeat(String customer) {
        if (seatsLeft > 0) {
            System.out.println("Seat booked for " + customer + " | Seats remaining: " + (--seatsLeft));
            return true;
        } else {
            System.out.println("Sold out! Booking rejected for" + customer);
            return false;
        }
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        CinemaBooking cinema = new CinemaBooking();
        Thread[] buyers = new Thread[8];
        for (int i = 0; i < buyers.length; i++) {
            final String user = "User-" + (i + 1);
            buyers[i] = new Thread(() -> cinema.bookSeat(user));
            buyers[i].start();
        }
        for (Thread t : buyers) t.join();
    }
}
```

Output Screenshot:



Learning Outcome:

- Protected limited shared resources against concurrent overselling.
- Applied monitor locks to guarantee serialized access to critical sections.
- Analyzed the behavior of competing concurrent threads under contention.
- Maintained data integrity in ticket-reservation architectures.
- Implemented safe state updates within synchronized transactional boundaries.